1. Difference between frontend, backend, and full-stack development

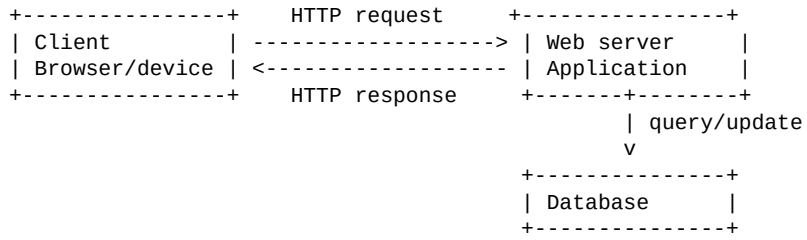
Frontend development: Builds the part users see and interact with. HTML provides structure, CSS controls presentation, and JavaScript adds interactivity. Example: a shopping website's product cards and cart button.

Backend development: Handles server-side logic, validation, authentication, and data processing. Example: checking login credentials or calculating an order total.

Full-stack development: Works across both frontend and backend, including connecting to databases and APIs. Example: building the shopping page and the service that returns products and saves orders.

In short: frontend = user interface; backend = server-side logic; full-stack = both.

2. Diagram showing the client-server model



The client requests a resource. The server processes the request, may read or write database data, and sends a response such as HTML, CSS, JavaScript, JSON, or an image.

3. How a browser requests and displays a web page

1. The user enters a URL or clicks a link.
2. DNS helps the browser find the server's IP address.
3. The browser establishes a connection; HTTPS also establishes encrypted TLS communication.
4. The browser sends an HTTP request. The server returns an HTTP response and a status code.
5. The browser parses HTML into the DOM and downloads linked CSS, JavaScript, images, and other resources.
6. It calculates styles and layout, paints the page, and displays it. JavaScript may update the page afterward.

4. Tools required for a web development environment

- **Computer/operating system:** Runs development applications.
- **Visual Studio Code:** Edits HTML, CSS, and JavaScript.
- **Web browser:** Tests and displays the web page.
- **Browser Developer Tools:** Inspects HTML/CSS, errors, and network requests.
- **Git:** Tracks code changes and versions.
- **GitHub:** Hosts repositories online and supports collaboration and review.
- **Live Server or local server:** Previews files through a local web address.
- **Node.js/npm (optional for basic HTML):** Runs JavaScript tooling and manages packages when a project needs them.

5. What is a web server? Give examples

A web server is software that receives HTTP/HTTPS requests and sends responses. It may deliver static files or forward requests to application code.

- **Apache HTTP Server:** Widely used open-source web server.
- **Nginx:** Web server, reverse proxy, and load balancer.
- **Microsoft IIS:** Web server for Windows environments.
- **Caddy:** Web server with simple configuration and automatic HTTPS.
- **Node.js application servers:** Servers built with JavaScript frameworks such as Express.

6. Roles of frontend developer, backend developer, and database administrator

- **Frontend developer:** Builds responsive, accessible user interfaces and connects them to APIs.
- **Backend developer:** Builds APIs, business logic, authentication, validation, and server-side processing.
- **Database administrator (DBA):** Manages database security, access, backups, recovery, performance, and availability.

Example: in a college registration system, the frontend builds the form, the backend validates and processes submissions, and the DBA manages the stored student and course records.

7. Install and configure VS Code for HTML, CSS, and JavaScript

1. Download Visual Studio Code from https://code.visualstudio.com/ and install it.

2. Open VS Code and choose File → Open Folder to open your project.

3. Create and save an index.html file. You can type ! and press Tab/Enter to insert an HTML template in supported setups.

4. If required, create style.css and script.js. Link them in HTML using a stylesheet link element and a script element.

5. Optionally install the Live Server extension to preview your page in a browser.

6. Use the browser Developer Tools to inspect layout and errors.

Screenshot requirement: Take a genuine screenshot showing VS Code open with your project and HTML file. Add that screenshot to the submission; this PDF cannot supply a screenshot of your computer.

8. Difference between static and dynamic websites

Static website: Usually serves prewritten files with the same content until the files are changed. Example: a basic HTML portfolio.

Dynamic website: Generates or personalizes content using user input, application logic, or stored data. Example: a banking dashboard showing the signed-in user's transactions.

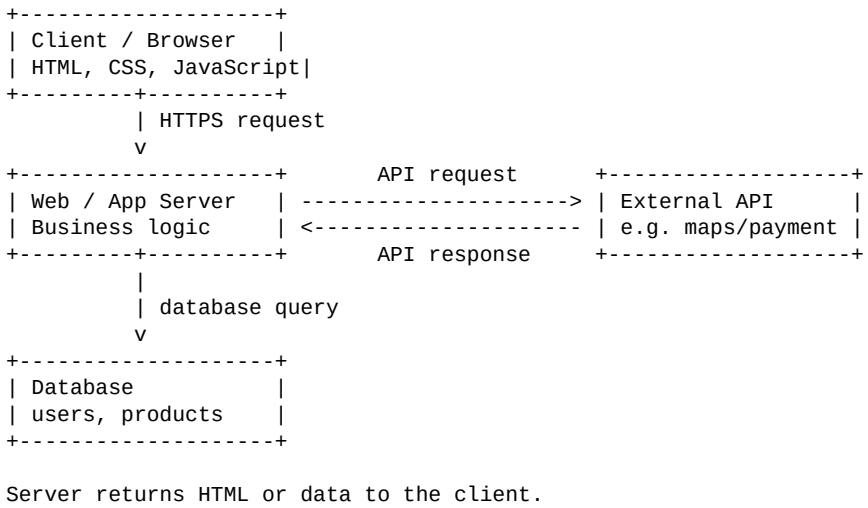
A static page can still use client-side JavaScript; the distinction is mainly how content is generated or personalized.

9. Five web browsers and their rendering engines

- **Google Chrome:** Blink rendering engine; V8 JavaScript engine.
- **Microsoft Edge:** Current Chromium-based versions use Blink and V8.
- **Opera:** Current mainstream Chromium-based versions use Blink and V8.
- **Mozilla Firefox:** Gecko rendering engine; SpiderMonkey JavaScript engine.
- **Apple Safari:** WebKit rendering engine; JavaScriptCore engine.

Rendering engines interpret HTML and CSS and determine how the page is drawn. Chrome, Edge, and Opera share Chromium-based technology, while Firefox uses Gecko and Safari uses WebKit. Details can vary by browser version and platform.

10. Labeled diagram of basic web architecture: client, server, database, APIs



The client communicates with the application server over HTTP/HTTPS. The server queries the database and may call external APIs for services such as maps, payments, or email. APIs define how components exchange requests and responses.

Submission checklist

- ■ Review the answers and make sure you can explain the concepts.
- ■ Add your genuine VS Code screenshot for Question 7.
- ■ Check the name and details required by your instructor.
- ■ Submit the PDF to the correct classroom assignment.